Concept Formation by View Application

A second domain on shared foundations in CogLean

Jeff Shrager

November 2026

Draft for Comment

The question

- CogLean is designed as a general, Mathlib-like library of cognitive processes: build *any* model on it, with proofs *and* executability.
- The CogLean paper's first worked example: **multi-digit arithmetic**.
- One example can't show the design worked. **The test:** a second model, in a very different domain.

Take CogLean to concept formation: how people understand something new by applying concepts they already have.

Forming concepts from old ones

- Concepts live inside theories (Murphy & Medin 1985); they change by restructuring (Carey 1985; Rumelhart & Norman 1976).
- Analogy maps a known structure onto a new one (Gentner 1983); frames supply defaults (Minsky 1975).
- Learners build mental models of devices (Kieras & Bovair 1984).
- **View application** (Shrager 1987): a learner exploring a device with no instructions (Shrager & Klahr 1986) revises its theory by merging in a *view*, a schema of a familiar kind of thing.

Left open in 1987: **which** view is brought in, and **when** the theory is revised.

View application in CogLib

Theories and views are **knowledge nets**: semantic nets over terms, the same representation as CogLib's concepts and semantic memory.

1. copy the view; 2. merge the theory in (match types; at random where ambiguous);
3. coerce re-parented instances; 4. bind holes; 5. one function per object.

Proved for any view and theory:

- the view is kept; **the view wins** on matched frames (non-monotonic change);
- unmatched theory frames survive; instances of matched types are re-parented.

Already in the library: BigTrak's Table 4 reproduced exactly (probability $1/2$).

A concept result already there: number concepts

- Prototype concepts (*even*, *multiple of 3*) combined by selective modification (Smith et al. 1988), graded membership (Hampton 2007): 14 is an *even multiple of 3* to degree $\approx .80$. Over-extension.
- Apply an **ARITHMETIC view**:
  - *even multiple of 3* becomes exactly divisibility by 6, for every number;
  - no over-extension anywhere; **typicality unchanged**, for every number.
- So typicality and membership **dissociate**, as Armstrong, Gleitman & Gleitman (1983) found for *odd number*; impossible before the view.

Which view? Retrieval through the shared memory

- Already: **spreading activation** (Collins & Loftus 1975). The same "+" key: *adds* with probability 2/3 for a naive learner; *appends a step* with 3/5 for one whose theory has a memory.
- New: **retrieval**. A cue (a key's label, the display) is coded and recognized by a memory, through CogLib's `Retriever` interface, the one the arithmetic model used. The stored view is applied.

For any memory:

- a cue confused with another brings in the other's view;
- a memory with the law of **determinacy** brings in one view per cue, at every attempt.

EPAM's net has determinacy at every setting; ACT-R's memory at none (the memory matrix).

When to revise? The decision cycle, again

Learning by revision	Decision cycle (as in the arithmetic solver)
theory, memory, episodes to explain	the state
accept the prediction	`confirm`: precondition, it's right
a failed prediction	**an impasse**
revise, prompted by a cue	**a repair**
which cues to consult	repair heuristics
don't consult a cue twice per failure	a critic

Not a new control structure: the decision cycle that ran the arithmetic backup.

In words: the next slide

The loop asks a learner for only three things:

- how a theory **predicts** the response to a sequence of actions;
- how a cue **revises** a theory: any process, any memory, a distribution of results;
- which **cues** an episode offers.

No knowledge nets, no views, no devices. Proved for every such learner:

- an impasse is exactly a failed prediction;
- a theory that predicts every episode is never revised.

The learner, in Lean

```
structure Reviser (Θ M Act Resp C B : Type) where
  predicts : Θ → List Act → Option Resp
  revise   : C → Θ → M → Dist (Θ × M × B)
  cues     : List Act × Resp → List C
  once     : Bool
```

- `CogLib/Process/TheoryRevision`: the loop, for any `Reviser`.
- `CogLib/Process/ViewRevision`: view application is *one* reviser.
- Others (rule induction; hypothesis-space search, Klahr & Dunbar 1988) can plug in and inherit the loop's theorems.

A phenomenon that transfers

The CogLean paper's **confusion-error pattern**, stated for arithmetic, apart from any mechanism:

- the familiar q is never answered wrongly by retrieval;
- every retrieved answer to the confused x is wrong;
- x falls back exactly as often as q.

For view retrieval (answer = the revised theory's prediction; fallback = exploration), the **same predicate holds**, given:

- both cues call up the same stored view, for certain;
- that view predicts q correctly and x wrongly.

Confident misinterpretation. A different mechanism; the phenomenon carried over unchanged.

New phenomena: garden paths and perseveration

- **Garden path**: the first interpretation adopted is later replaced.
- **Perseveration**: the same interpretation adopted at two successive failures.

Both stated over the sequence of interpretations, naming no mechanism.

Perseveration, for a class of memories. For *every* memory with determinacy (unchanged by revising): revisions by a given cue bring in one and the same retrieval, every time, from every theory.

- A learner who keeps consulting that cue keeps re-applying a failed interpretation: a relative of *Einstellung* (Luchins 1942).
- ACT-R's memory lacks the law and can escape.
- Sufficient, not necessary; and not a claim that people's memories have the law.

Views from parts

- SET-A-TIME bundles two ideas. Decomposed:
  - **SET-A-NUMBER**: the key enters a number;
  - **NUMBER-IS-A-TIME** / **NUMBER-IS-A-DURATION**: what the number means.
- Each mentions one slot of the key's type. View application's merge lets a view win only on what it mentions, so a later view **keeps** what an earlier one said about other slots.
- In the microwave: one part predicts nothing; two compose, in either order.
- A revision can replace the meaning and **keep** "enters a number": tuning, not restructuring.

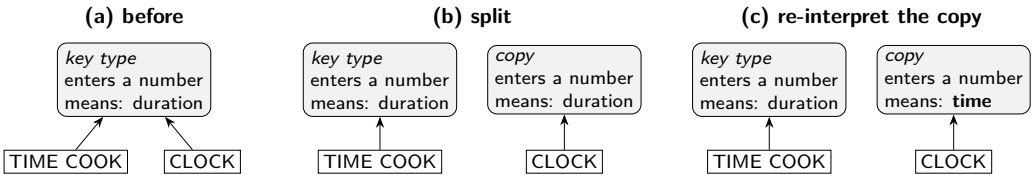
(Commutation of views on disjoint slots: certified in instances; open in general.)

Why concepts must split

- Views act on **types**: every instance of a type changes together.
- Proved: actions of one type are **read alike**.
- So, for *every* theory in which TIME COOK and CLOCK share a type and both enter a number:
  9 0 START is predicted alike after either, and **no such theory predicts both** "cooks 90 s" and "invalid time".

No view, and no amount of revising, fixes this. The concept has to **differentiate**.

A split, step by step



- (a) CLOCK fails; re-interpreting the shared type would break TIME COOK.
- (b) Copy the type for CLOCK: **nothing predicts differently yet**.
- (c) Aim NUMBER-IS-A-TIME at the copy: CLOCK changes, TIME COOK doesn't.
- A split isn't free (one more type to keep track of), so the learner splits only when forced.

Differentiation: split, then re-interpret

- **Split** copies an action's type and moves the action to the copy. Proved:
  - a split **changes no prediction** (it's conservative);
  - it **separates** the action from its old type's other instances.
- **When**: if re-interpreting the failing key's type would break an explained episode, split first.

Episodes: TIME COOK, CLOCK, TIME COOK (EPAM; 12 cycles).

Learner	knows both	finishes	oscillates
cannot split	0	5/6	5/6
splits on conflict	5/6	5/6	0

What was shared, and what was new

CogLib component	Arithmetic	Concepts
Exact distributions; `bind`	yes	yes
Retriever interface; EPAM; ACT-R	yes	yes
Laws of memory	yes	yes
`ConfusionErrorPattern`	yes	yes
Decision cycle, impasses, repairs	solver	revision loop
Knowledge nets; view application	—	yes
New, general: theory execution, view retrieval, the revision loop, view composition, type splitting; garden paths, perseveration, oscillation	—	new

Reused by *citing proofs*, not by imitation. The device is data in its own package.

A worked case: learning a microwave

- Keys TIME COOK, CLOCK, 9, 0, START. After TIME COOK, 9 0 START cooks 90 seconds; after CLOCK, 90 is an invalid time.
- The learner understands digits and START; TIME COOK and CLOCK: unknown. It's at an impasse at once.
- Views: **SET-A-TIME**, **DURATION**. Prior knowledge: a VCR's SET TIME (with SET-A-TIME), a kitchen timer (with DURATION).
- EPAM's net notices the countdown first: **both labels call up the VCR**. ACT-R: graded.

	EPAM	ACT-R
label + display	garden path when the label comes first; always solved	garden paths rarer; usually solved
label only	**certain perseveration**; never solved	escapes; usually solved

What it says about concept formation

- **Which prior concept is recruited is a memory question.** Properties of memory become predictions: determinacy $\Rightarrow$ perseveration; confusion $\Rightarrow$ confident misinterpretation.
- **When to revise is problem solving**: the impasse structure, connecting theory change to repair theory and Soar.
- **Concepts must sometimes split.** Views act on types, so shared types can't be told apart (proved). Differentiation is necessary, and conservative: the split changes nothing, later re-interpretation does (Carey; Thagard).

Limitations, and next

Limitations

- Memory doesn't learn during revision.
- Splitting is conflict-driven, not weighed against overwriting.
- View commutation proved only in instances.
- An illustrative device; no data.
- ACT-R without retrieval as use.
- Parts of the 1987 learner not formalized.

Next

- Learning which views succeed for which cues.
- Choosing between splitting and overwriting.
- A second reviser on the same loop.
- Contact with data: instructionless-learning protocols (Shrager & Klahr 1986).

Take-home

- CogLean: a general library; arithmetic was only its first example. For concept formation, it supplied:
  - the memory interface and its laws;
  - a phenomenon first stated in the arithmetic example;
  - the decision cycle.
- The new parts went into the library in general form: view retrieval, the revision loop, view composition, type splitting.
- Differentiation: necessary when types are shared (proved), conservative, and what ends oscillation.
- Results at every level: for every learner; for every memory with a law; at a setting, as witnesses.
- Complementary to architectures (Soar, ACT-R, the Common Model): it accumulates proved results about mechanisms, across architectural commitments.

That's what a cumulative library should do.